

Craftboard

A Real-Time Collaborative Design Canvas

Project Overview & Technical Documentation

Version 1.0 | August 13, 2026
Prepared by: Development Team

1 Executive Summary

Craftboard is a web-based, real-time collaborative design canvas that lets distributed teams ideate and design together on a shared, structured whiteboard. It combines a Konva-powered canvas frontend with a PostgreSQL-backed Express API, Liveblocks for real-time synchronization and presence, Clerk for authentication, and a locally hosted Ollama AI assistant that turns natural-language prompts into ready-to-place design layouts.

The product is designed as a lightweight, self-hostable alternative to heavyweight proprietary design tools. Because the AI inference runs on a local Ollama instance rather than a third-party cloud API, teams keep full control over their design data, avoid per-seat SaaS costs, and retain offline-capable operation.

This document summarizes what was built, how the system works, the technology choices behind it, the verification performed, and the recommended path forward.

2 Problem Statement

Modern product and design teams frequently struggle with three related challenges:

- Collaboration friction: sharing designs through static screenshots and file email chains creates version confusion and a lack of a single source of truth.
- Heavyweight tooling: mainstream design platforms are either proprietary SaaS with rising per-seat pricing, or overly complex self-hosted stacks that are expensive to operate.
- Privacy and cost of AI: AI-assisted design is usually gated behind cloud APIs that raise data-privacy concerns, add latency, and incur usage fees.

Craftboard addresses all three by providing a real-time, role-based collaborative canvas with a structured shape model, self-hosted infrastructure, and a local AI design assistant.

3 Solution Overview

Craftboard organizes work into projects, each containing one or more pages (canvases). Users sign in through Clerk, are grouped into teams, and are assigned roles that control what they may do on a project (owner, editor, viewer). Any change to a canvas is synchronized to other connected clients in real time through Liveblocks, and asynchronously persisted to PostgreSQL.

A built-in access-management workflow lets users request access to a project and lets the owner or an editor approve or deny those requests from the dashboard. A Socket.IO layer delivers real-time notifications for these events.

The AI assistant, served by an Express route and powered by a local Ollama deployment, accepts a plain-language instruction such as "build a hero section with a navy background and a call-to-action button" and returns a structured set of shapes that the client renders directly onto the canvas.

4 How It Works

4.1 User Journey

A visitor lands on the public marketing site, reviews the feature set and architecture diagram, and signs up with email, Google, or GitHub via Clerk. After sign-in the user is redirected to the dashboard, which lists projects, recent activity, and pending access requests.

The user creates a project, then a page inside it, and opens the editor. On the canvas they can add and edit over a dozen structured shape types, move and resize them, and delete them. If they have been granted edit access, they can invite collaborators by email or link.

To bootstrap a layout, the user opens the AI assistant panel, enters a description, and chooses to have the generated shapes added to the canvas for refinement. When the user leaves the page, all changes are persisted to the database.

4.2 Real-Time Synchronization Model

Each page is backed by a dedicated Liveblocks room. Connected clients keep a local copy of the room state, apply optimistic updates instantly, and reconcile with peers through Liveblocks' conflict-free data structures. Live cursor positions and presence data provide the live-collaboration feel.

Persistence is intentionally decoupled from the real-time path. A Liveblocks `storageUpdated` webhook is delivered to the Express API and written to PostgreSQL, so the database is never on the critical path of an interactive edit. The client also fetches room initial state on entry so a user joining a session receives the current design immediately.

5 Key Features

- Structured design canvas: 18 shape types covering common design blocks (buttons, headings, text, images, cards, lists, and more) with properties for labels, colors, and dimensions.
- Real-time collaboration: live object edits, presence, and cursor awareness via Liveblocks.
- Project hierarchy: teams own projects; projects contain pages; pages contain shapes.
- Role-based access control: owner / editor / viewer permissions enforced server-side.
- Invite and access-request workflows with real-time notifications.
- AI layout generation: local Ollama-powered assistant that returns structured, editable shapes.
- Public marketing site with a rendered system-architecture diagram.
- Swagger-documented REST API with a health-check endpoint.
- Retry-capable HTTP client and graceful fallback when the AI service is unavailable.

6 Technology Stack

Layer	Technology	Purpose
Frontend	React 19, Vite, TypeScript	Application UI and build tooling
Canvas	Konva / react-konva	Rendering and interaction of design shapes
Real-time sync	Liveblocks (React) + @liveblocks/node	Rooms, presence, CRDT state, webhooks
Notifications	Socket.IO (client + server)	Real-time access events and updates
Backend	Node.js, Express 5, TypeScript	REST API, business logic, AI proxy
Data store	PostgreSQL + Prisma ORM	Persistent storage of users, teams, projects, pages, shapes
Authentication	Clerk	Sign-in, JWT issuance and verification
AI	Express route + local Ollama (qwen2.5)	Structured layout generation from prompts
Docs / API	Swagger (swagger-ui-express)	Interactive API reference

7 System Architecture

The system is organized into four logical layers. External cloud services (Liveblocks, Clerk) handle real-time sync and identity. The client is a React single-page application with a dashboard, a marketing landing page, and the canvas editor. The application server exposes the REST API, verifies JWTs, enforces role-based access, proxies AI requests to Ollama, and consumes Liveblocks webhooks. Data stores consist of PostgreSQL for durable records and the local Ollama model service for AI inference.

Requests flow through a small set of well-defined surfaces: the API receives authenticated HTTP calls; Liveblocks pushes storageUpdated webhooks to the server; Socket.IO carries real-time notifications to clients; and Ollama serves model inference to the AI route. This separation keeps responsibilities clean and makes each layer independently testable and scalable.

8 Security & Access Control

- Authentication: all protected routes require a valid Clerk JWT, verified by a `requireAuth` middleware before any handler runs.
- Authorization: a `requireProjectRole` middleware enforces a three-level hierarchy — owner (3), editor (2), viewer (1) — and rejects insufficient-privilege requests with 403.
- Access requests: a request-access endpoint lets a user ask for access to a project; only owners can approve or deny, and the dashboard surfaces pending requests.
- Webhook security: Liveblocks webhook payloads are verified against the configured secret.
- CORS: the API restricts origins to the allowed client origins rather than accepting all requests.
- Input hygiene: AI prompt payloads are validated and length-clamped before being sent to Ollama.
- Credential management: secrets are supplied through environment variables and are not committed to the repository.

9 Testing & Quality Assurance

Verification covered the critical user journeys end to end, from a fresh sign-in through canvas editing, collaboration, AI generation, and access management. The table below records the test scenarios and their outcomes.

ID	Area	Scenario	Expected Result	Status
T-01	Auth	User signs in with email via Clerk	JWT issued; redirect to dashboard; profile shown	Passed
T-02	Auth	Unauthenticated request to protected API route	Request rejected with 401	Passed
T-03	API	Create project	Project persisted and returned with UUID and timestamps	Passed
T-04	API	List projects for the signed-in user	Paged list with search and pagination metadata	Passed
T-05	API	Create a page within a project	Page persisted and linked to project (200)	Passed
T-06	API	List pages with pagination	Page data returned with pagination metadata	Passed
T-07	API	List project members with roles	Members with role levels returned (200)	Passed
T-08	Canvas	Add, move, resize, and delete shapes	Canvas updates instantly and persists	Passed
T-09	Collaboration	Two clients edit the same page simultaneously	Changes synchronize live; presence visible	Passed
T-10	Collaboration	Late joiner opens an existing page	Current room state loaded immediately	Passed
T-11	AI	Prompt "hero section..."	Valid structured shape JSON returned and rendered	Passed
T-12	AI	Ollama unavailable	Graceful error with status surfaced to user	Passed
T-13	Access	Request access to a project	Pending request visible to project owner	Passed
T-14	Access	Owner approves / denies a request	Role updated or request rejected with notification	Passed
T-15	Security	Viewer attempts edit-only action	Request rejected with 403 by server	Passed
T-16	Build	Server type-check (tsc)	No type errors	Passed
T-17	Build	Production client build (vite build)	Build completes without errors	Passed
T-18	Site	Landing page renders architecture diagram	Diagram and sections render correctly	Passed

10 Deployment & Operations

- Prerequisites: Node.js, PostgreSQL, an Ollama instance serving a chat model, and accounts for Liveblocks and Clerk.
- Configuration: all keys and database credentials are supplied via a .env file documented in the repository; no secrets are stored in source control.
- Local development: the API runs with ts-node in watch mode, the client runs through Vite with a proxy to the API, and the database schema is managed with Prisma migrations.
- Health monitoring: a /api/health endpoint reports server and database status for uptime checks.
- API documentation: a Swagger UI is served alongside the API for consumers and QA.

11 Performance & Scalability Considerations

- Persistence is asynchronous: webhook-based writes keep the database off the interactive editing path.
- The canvas relies on Konva's layered rendering, which comfortably handles hundreds of shape nodes.
- The editor is lazy-loaded so the dashboard and landing page stay light.
- The HTTP client retries transient failures, reducing spurious user-facing errors.
- AI inference is local, so generation cost is bounded by hardware and incurs no per-call cloud fees.
- Rooms are isolated per page, so collaboration load scales horizontally with concurrent editing sessions.

12 Roadmap

- Add caching (e.g., Redis) for high-read paths such as the dashboard and project listing.
- Introduce a work queue for webhook processing and notifications to absorb bursts under load.
- Extract a shared types package to keep client and server contracts synchronized.
- Extend the AI assistant to image generation and richer layout presets.
- Expand automated test coverage (unit and integration) to lock in current behavior.
- Add export of canvases to PNG/SVG and support for page templates.

Appendix A API Endpoint Reference

The REST API is grouped by domain: authentication, projects, pages, members, access requests, notifications, and AI. Key routes include:

Method	Route	Description
GET	/api/health	Server and database health check
GET/POST	/api/projects	List or create projects
GET/PATCH/DELETE	/api/projects/:projectId	Fetch, update, or delete a project
GET/POST	/api/projects/:projectId/pages	List or create pages
GET	/api/projects/:projectId/members	List project members with roles
POST	/api/access-request	Request access to a project
GET/POST	/api/notifications	List or create notifications
POST	/api/ai/generate	Generate structured layout from a prompt